

Infraestructuras Verdes

Grupo de Investigación AGE-VITAL

17 de marzo de 2026

1. Introducción

Este proyecto tiene como objetivo el desarrollo de un sistema de monitoreo y control para infraestructuras verdes, específicamente enfocado en techos verdes. El sistema permite medir variables ambientales y del suelo, además de controlar el riego de manera automatizada mediante una electroválvula.

2. Objetivos

- Monitorear variables ambientales (temperatura y humedad).
- Medir condiciones del suelo.
- Controlar el flujo de agua mediante una válvula.
- Enviar datos a un servidor para análisis.

3. Arquitectura del Sistema

El sistema está basado en un microcontrolador ESP32 TTGO, el cual integra:

- Sensores de temperatura y humedad
- Sensor de flujo
- Sensor de suelo (Modbus RS485)
- Pantalla TFT
- Control de válvula
- Conectividad WiFi

4. Diagrama de Conexiones

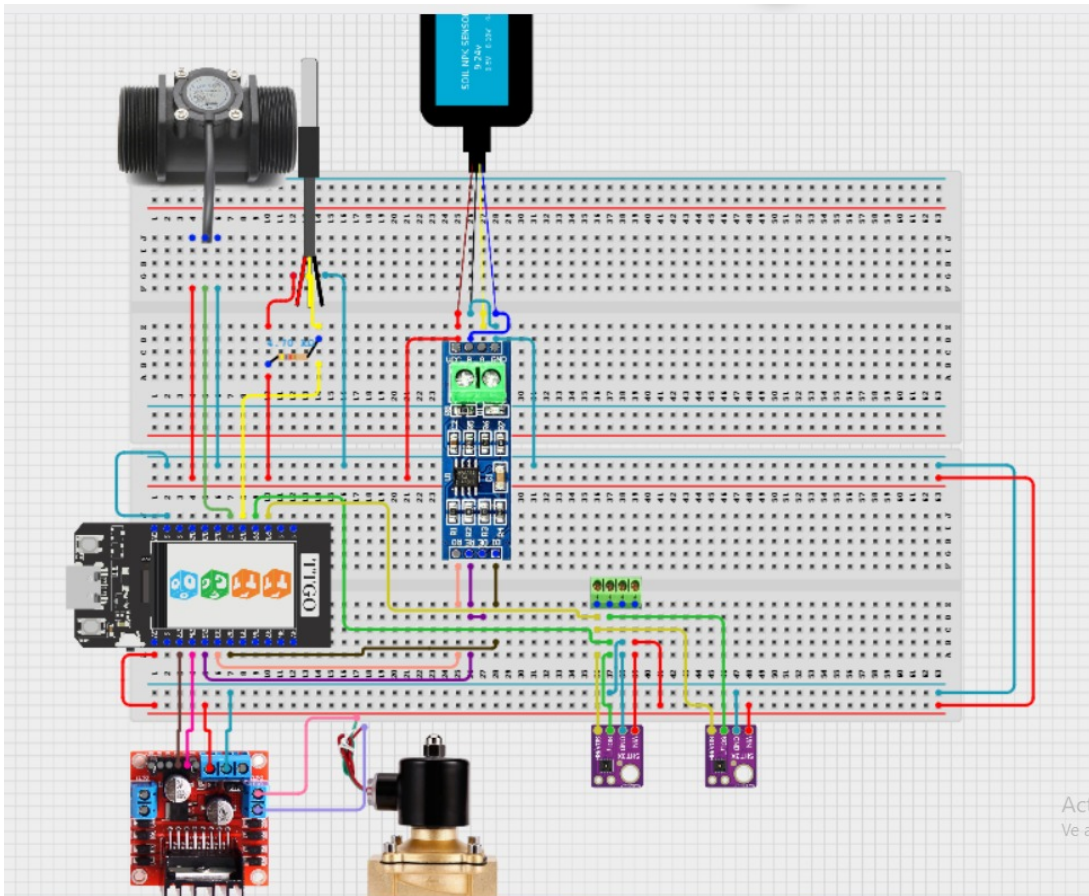


Figura 1: Diagrama físico del sistema

[Ver diseño en Círculo Designer](#)

5. Asignación de Pines

5.1. Pantalla TFT

- CS: GPIO 5
- RST: GPIO 23
- DC: GPIO 16
- MOSI: GPIO 19
- SCLK: GPIO 18
- BL: GPIO 4

5.2. Sensor RS485 (Humedad y temperatura de suelo)

- DE/RE: GPIO 25
- RX: GPIO 33
- TX: GPIO 32

5.3. Sensor OneWire (DS18B20)

- DATA: GPIO 17

5.4. Sensor SHT31 (I2C)

- SDA: GPIO 21
- SCL: GPIO 22

5.5. Sensor de flujo

- Señal: GPIO 2

5.6. Electroválvula

- Control: GPIO 26

6. Descripción de Sensores

6.1. Sensor SHT31

Sensor digital de alta precisión que mide:

- Temperatura del aire
- Humedad relativa

Comunicación mediante protocolo I2C.

6.2. Sensor DS18B20

Sensor digital de temperatura:

- Alta precisión
- Comunicación OneWire

6.3. Sensor de suelo RS485

Sensor industrial que mide:

- Temperatura del suelo
- Humedad del suelo

Comunicación mediante protocolo Modbus RTU.

6.4. Sensor de flujo

Sensor basado en efecto Hall:

- Genera pulsos proporcionales al flujo de agua
- Relación: 288 pulsos por litro

7. Control de la Válvula

La válvula es controlada mediante una salida digital del ESP32. Cuando se activa:

- Se inicia el conteo de flujo
- Se calcula el volumen de agua utilizado

8. Comunicación

El sistema envía datos mediante HTTP a un servidor Orion Context Broker en formato JSON.

9. Código del Sistema

```
#include <WiFi.h>
#include <HTTPClient.h>
#include <ArduinoJson.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_ST7789.h>
#include <SPI.h>
#include <ModbusMaster.h>
#include <OneWire.h>
#include <DallasTemperature.h>
#include <Adafruit_SHT31.h>

// ----- WIFI -----
const char* ssid      = "F_TENO";
const char* password = "Osteca52";

// Orion sin headers FIWARE
const char* serverUrl = "http://192.168.1.55:1026/v2/entities/
    Techos_verdes_data/attrs";

// ----- PANTALLA -----
#define TFT_CS    5
#define TFT_RST   23
#define TFT_DC    16
#define TFT_MOSI  19
#define TFT_SCLK  18
```

```

#define TFT_BL      4

Adafruit_ST7789 tft = Adafruit_ST7789(TFT_CS, TFT_DC, TFT_MOSI,
    TFT_SCLK, TFT_RST);

// ----- SENSORES -----
#define MAX485_DE 25
#define RX2_PIN   33
#define TX2_PIN   32
ModbusMaster node;

#define ONE_WIRE_BUS 17
OneWire oneWire(ONE_WIRE_BUS);
DallasTemperature sensors(&oneWire);

Adafruit_SHT31 sht31 = Adafruit_SHT31();

// ----- FLUJO Y V LVULA -----
#define FLOW_PIN 2
#define VALVE_PIN 26

volatile long pulsosFlujo = 0;
void IRAM_ATTR contarPulsos() { pulsosFlujo++; }

const float PULSOS_POR_LITRO = 288.0;

float flow_litros_sesion = 0;
float flow_litros_total = 0;
bool valve_state = false;

// ----- TIMERS -----
unsigned long lastReadTime = 0;
unsigned long lastSendTime = 0;
unsigned long lastScreenTime = 0;

const unsigned long readInterval = 2000;
const unsigned long sendInterval = 10000;
const unsigned long screenInterval = 5000;

// ----- VARIABLES SENSORES -----
float soil_temp_1 = 0;
float soil_humidity_1 = 0;
float soil_temp_2 = 0;
float air_temp_low = 0;
float air_humidity_low = 0;
float air_temp_high = 0;
float air_humidity_high = 0;

float prev_soil_temp_1 = -999;
float prev_soil_humidity_1 = -999;
float prev_soil_temp_2 = -999;

```

```

float prev_air_temp_low      = -999;
float prev_air_humidity_low  = -999;
float prev_air_temp_high     = -999;
float prev_air_humidity_high = -999;
float prev_flow_session      = -999;
bool  prev_valve_state       = false;

// ----- RS485 -----
void preTransmission() { digitalWrite(MAX485_DE, 1); delay(2); }
void postTransmission() { delay(2); digitalWrite(MAX485_DE, 0); }

// ----- COLORES -----
#define COLOR_BG          ST77XX_BLACK
#define COLOR_TITULO_BG   ST77XX_CYAN
#define COLOR_TITULO_FG   ST77XX_BLACK
#define COLOR_LABEL        0x7BEF
#define COLOR_TEMP         ST77XX_YELLOW
#define COLOR_HUM          0x07FF
#define COLOR_SEP          0x4208
#define COLOR_IP_OK        ST77XX_GREEN
#define COLOR_IP_ERR       ST77XX_RED
#define COLOR_ROW_ALT      0x0841
#define COLOR_FLOW         ST77XX_MAGENTA
#define COLOR_VALVE_ON     ST77XX_GREEN
#define COLOR_VALVE_OFF    ST77XX_RED

#define FILA_Y_START 17
#define FILA_H        22

// ----- V LVULA -----
void setValvula(bool abrir) {
    valve_state = abrir;
    digitalWrite(VALUE_PIN, abrir ? HIGH : LOW);

    if (abrir) {
        noInterrupts();
        pulsosFlujo = 0;
        interrupts();
        flow_litros_session = 0;
        Serial.println("Valvula ABIERTA - sesion de riego iniciada");
    } else {
        Serial.printf("Valvula CERRADA - litros esta sesion: %.2f\n",
            flow_litros_session);
    }
}

// ----- PANTALLA -----
void actualizarValorFila(int fila, float valorNuevo, float &
    valorPrev, const char* unidad, uint16_t colorValor) {
    if (abs(valorNuevo - valorPrev) < 0.05) return;
    valorPrev = valorNuevo;

```

```

int y = FILA_Y_START + fila * FILA_H;
tft.fillRect(4, y + 10, 125, 11, (fila % 2 == 0) ?
    COLOR_ROW_ALT : COLOR_BG);

tft.setTextSize(1);
tft.setTextColor(colorValor);
tft.setCursor(4, y + 11);
tft.print(valorNuevo, 2);

tft.setTextColor(COLOR_LABEL);
tft.setCursor(60, y + 11);
tft.print(unidad);
}

void dibujarEstructura() {
    tft.fillScreen(COLOR_BG);

    tft.fillRect(0, 0, 135, 15, COLOR_TITULO_BG);
    tft.setTextColor(COLOR_TITULO_FG);
    tft.setTextSize(1);
    tft.setCursor(12, 4);
    tft.print("TECHOS_VERDES");
    tft.drawFastHLine(0, 15, 135, COLOR_SEP);
    tft.drawFastHLine(0, 16, 135, COLOR_TITULO_BG);

    const char* labels[] = {
        "Suelo_Temp_1",    "Suelo_Hum_1",
        "Suelo_Temp_2",    "Aire_Temp_Low",
        "Aire_Hum_Low",    "Aire_Temp_High",
        "Aire_Hum_High",   "Flujo_Sesion",
        "Valvula"
    };

    for (int i = 0; i < 9; i++) {
        int y = FILA_Y_START + i * FILA_H;
        tft.fillRect(0, y, 135, FILA_H, (i % 2 == 0) ? COLOR_ROW_ALT
            : COLOR_BG);
        tft.setTextSize(1);
        tft.setTextColor(COLOR_LABEL);
        tft.setCursor(4, y + 2);
        tft.print(labels[i]);
    }
}

void actualizarPantalla() {
    actualizarValorFila(0, soil_temp_1,        prev_soil_temp_1,
        "C", COLOR_TEMP);
    actualizarValorFila(1, soil_humidity_1,    prev_soil_humidity_1,
        "%", COLOR_HUM);
}

```

```

actualizarValorFila(2, soil_temp_2,          prev_soil_temp_2,
                  "C", COLOR_TEMP);
actualizarValorFila(3, air_temp_low,         prev_air_temp_low,
                  "C", COLOR_TEMP);
actualizarValorFila(4, air_humidity_low,     prev_air_humidity_low, "%", COLOR_HUM);
actualizarValorFila(5, air_temp_high,        prev_air_temp_high,
                  "C", COLOR_TEMP);
actualizarValorFila(6, air_humidity_high,    prev_air_humidity_high, "%", COLOR_HUM);
actualizarValorFila(7, flow_litros_sesion,   prev_flow_sesion,
                  "L", COLOR_FLOW);

if (valve_state != prev_valve_state) {
    prev_valve_state = valve_state;
    int y = FILA_Y_START + 8 * FILA_H;
    tft.fillRect(4, y + 10, 125, 11, (8 % 2 == 0) ? COLOR_ROW_ALT
                : COLOR_BG);
    tft.setTextSize(1);
    tft.setTextColor(valve_state ? COLOR_VALVE_ON :
                    COLOR_VALVE_OFF);
    tft.setCursor(4, y + 11);
    tft.print(valve_state ? "ABIERTA" : "CERRADA");
}

int yPie = FILA_Y_START + 9 * FILA_H;
tft.fillRect(0, yPie, 135, 240 - yPie, COLOR_BG);
tft.setTextSize(1);
if (WiFi.status() == WL_CONNECTED) {
    tft.setTextColor(COLOR_IP_OK);
    tft.setCursor(4, yPie + 2);
    tft.print("WiFi OK");
    tft.setTextColor(COLOR_LABEL);
    tft.setCursor(4, yPie + 12);
    tft.print(WiFi.localIP().toString());
} else {
    tft.setTextColor(COLOR_IP_ERR);
    tft.setCursor(4, yPie + 2);
    tft.print("WiFi DESCONECTADO");
}
}

// ----- PAYLOAD -----
String construirPayload() {
    StaticJsonDocument<1536> doc;

    doc["soil_temp_1"]["type"]      = "Number";
    doc["soil_temp_1"]["value"]    = soil_temp_1;

    doc["soil_humidity_1"]["type"]  = "Number";
    doc["soil_humidity_1"]["value"] = soil_humidity_1;

```



```

doc["soil_temp_2"]["type"]      = "Number";
doc["soil_temp_2"]["value"]    = soil_temp_2;

doc["air_temp_low"]["type"]    = "Number";
doc["air_temp_low"]["value"]   = air_temp_low;

doc["air_humidity_low"]["type"] = "Number";
doc["air_humidity_low"]["value"] = air_humidity_low;

doc["air_temp_high"]["type"]   = "Number";
doc["air_temp_high"]["value"]  = air_temp_high;

doc["air_humidity_high"]["type"] = "Number";
doc["air_humidity_high"]["value"] = air_humidity_high;

doc["flow_litros_sesion"]["type"] = "Number";
doc["flow_litros_sesion"]["value"] = flow_litros_sesion;

doc["flow_litros_total"]["type"] = "Number";
doc["flow_litros_total"]["value"] = flow_litros_total;

doc["valve_state"]["type"]      = "Number";
doc["valve_state"]["value"]    = valve_state ? 1 : 0;

String payload;
serializeJson(doc, payload);
return payload;
}

// ----- ENVIAR A ORION -----
void enviarDatos() {
  if (WiFi.status() != WL_CONNECTED) {
    Serial.println("WiFi desconectado");
    WiFi.reconnect();
    return;
  }

  HTTPClient http;
  http.begin(serverUrl);
  http.addHeader("Content-Type", "application/json");

  String payload = construirPayload();

  Serial.println("\n--- Enviando a Orion ---");
  Serial.println(payload);

  int httpCode = http.POST(payload);
  Serial.printf("HTTP: %d\n", httpCode);

  if (httpCode == 204) {

```

```

        Serial.println("OK-▯datos▯actualizados▯en▯Orion");
    } else {
        Serial.printf("Error:▯%s\n", http.errorToString(httpCode).
            c_str());
        String response = http.getString();
        Serial.println("Respuesta▯del▯servidor:");
        Serial.println(response);
    }

    http.end();
}

// ----- SETUP -----
void setup() {
    Serial.begin(115200);

    pinMode(TFT_BL, OUTPUT);
    digitalWrite(TFT_BL, HIGH);
    tft.init(135, 240);
    tft.setRotation(2);
    tft.fillScreen(COLOR_BG);

    tft.setTextColor(COLOR_TITULO_BG);
    tft.setTextSize(2);
    tft.setCursor(8, 100);
    tft.print("Conectando");
    tft.setCursor(35, 120);
    tft.print("WiFi...");

    WiFi.begin(ssid, password);
    int intentos = 0;
    while (WiFi.status() != WL_CONNECTED && intentos < 20) {
        delay(500);
        Serial.print(".");
        intentos++;
    }

    if (WiFi.status() == WL_CONNECTED) {
        Serial.println("\nWiFi▯OK:▯" + WiFi.localIP().toString());
    } else {
        Serial.println("\nWiFi▯fallo");
    }

    pinMode(MAX485_DE, OUTPUT);
    digitalWrite(MAX485_DE, 0);
    Serial2.begin(9600, SERIAL_8N1, RX2_PIN, TX2_PIN);
    node.begin(1, Serial2);
    node.preTransmission(preTransmission);
    node.postTransmission(postTransmission);

    sensors.begin();

```

```

Wire.begin(21, 22);
if (!sht31.begin(0x44)) {
    Serial.println("SHT31 no encontrado");
} else {
    Serial.println("SHT31 OK");
}

pinMode(FLOW_PIN, INPUT_PULLUP);
attachInterrupt(digitalPinToInterrupt(FLOW_PIN), contarPulsos,
    RISING);

pinMode(VALVE_PIN, OUTPUT);
digitalWrite(VALVE_PIN, LOW);

dibujarEstructura();

Serial.println("Sistema listo.");
Serial.println("Comandos: '1' = abrir valvula | '0' = cerrar valvula");
}

// ----- LOOP -----
void loop() {
    unsigned long currentMillis = millis();

    // Control v lvula por Serial
    if (Serial.available()) {
        char cmd = Serial.read();
        if (cmd == '1') setValvula(true);
        if (cmd == '0') setValvula(false);
    }

    // Lectura cada 2 seg
    if (currentMillis - lastReadTime >= readInterval) {
        delay(50);

        uint8_t res = node.readHoldingRegisters(2, 2);
        if (res == node.ku8MBSuccess) {
            soil_humidity_1 = node.getResponseBuffer(0) / 10.0;
            soil_temp_1 = node.getResponseBuffer(1) / 10.0;
        } else {
            Serial.printf("Modbus error: 0x%02X\n", res);
        }

        sensors.requestTemperatures();
        soil_temp_2 = sensors.getTempCByIndex(0);
        if (soil_temp_2 == DEVICE_DISCONNECTED_C) soil_temp_2 = 0;

        air_temp_low = sht31.readTemperature();
        air_humidity_low = sht31.readHumidity();
    }
}

```

```

    if (isnan(air_temp_low))      air_temp_low = 0;
    if (isnan(air_humidity_low)) air_humidity_low = 0;

    air_temp_high      = air_temp_low;
    air_humidity_high = air_humidity_low;

    if (valve_state) {
        noInterrupts();
        long pulsos = pulsosFlujo;
        pulsosFlujo = 0;
        interrupts();

        float litros = pulsos / PULSOS_POR_LITRO;
        flow_litros_sesion += litros;
        flow_litros_total  += litros;
    }

    lastReadTime = currentMillis;
}

// Pantalla cada 5 seg
if (currentMillis - lastScreenTime >= screenInterval) {
    actualizarPantalla();
    lastScreenTime = currentMillis;
}

// Envío cada 10 seg
if (currentMillis - lastSendTime >= sendInterval) {
    enviarDatos();
    lastSendTime = currentMillis;
}
}

```

10. Funcionamiento General

El sistema opera en ciclos:

- Cada 2 segundos: lectura de sensores
- Cada 5 segundos: actualización de pantalla
- Cada 10 segundos: envío de datos

11. Conclusiones

El sistema desarrollado permite:

- Monitoreo en tiempo real
- Control eficiente del riego

- Integración con plataformas IoT

Esto lo convierte en una solución viable para aplicaciones de agricultura urbana y sostenibilidad.